Lab Handout 2-D
Packet Parser Finite State Machine

Capstone — Module 2: Field-Programmable Gate Array (FPGA) Register-Transfer Level (RTL) Design and Signal Ingestion

1 Objective

Delete the bring-up `frame_detector` from LH1-H, write a new `pkt_parser.vhd` that drains `byte_fifo`, locks onto the Module 1 interceptor forwarding frame (start delimiter `0x7E`, 1-byte length, 8-byte metadata, 32-byte embedded radio packet, 1-byte Cyclic Redundancy Check (CRC)-8), and produces registered outputs for each field. By the end of this handout a valid frame will emit a `frame_valid` pulse with the parsed fields stable on the same cycle. The parser is type-agnostic: the same code path handles DATA, ACK, and PAUSE radio packets because they share the 32-byte layout. Packet-type discrimination happens later in the feature extraction module (Lab Handout 2-F).

2 Prerequisites

- Lab Handout 2-C (`byte_fifo` replaced with the depth-64 Module 2 version).
- Module 1 receiver frame format reference (43 bytes total).

3 Background

The interceptor forwarding frame is defined by the Module 1 lesson plan (Table 6) and is the single canonical spec across Modules 1–5. Reproduced verbatim below:

Offset	Field	Size	Notes
0	Start delimiter `0x7E`	1 B	Frame sync
1	Length byte	1 B	Must equal `0x29` (41 decimal)
2–5	RX timestamp (uint32, LE, ms)	4 B	Metadata
6	RSSI (int8, dBm)	1 B	Metadata
7	SNR (int8, 0.25 dB)	1 B	Metadata
8–9	Frequency error (int16, LE, Hz)	2 B	Metadata
10–41	Radio packet (32 bytes; DATA / ACK / PAUSE)	32 B	Includes embedded CRC-16 at packet bytes 30–31 = frame 40–41
42	Frame CRC-8/ITU (poly `0x07`) over bytes 1–41	1 B	Transport integrity

Length-Field Rule

The length byte equals the count of bytes *excluding the start delimiter and the trailing CRC*. For a 43-byte frame, byte[1] must equal `0x29` (41 decimal). Reject any frame where byte[1] $\neq$ `0x29`.

Byte 42 is the frame CRC-8/ITU computed over bytes 1–41 (not 0–41 — the start delimiter is excluded). The radio packet also carries its own CRC-16/CCITT-FALSE internally at packet bytes 30–31 (frame offsets 40–41), checked in Lab Handout 2-E. **The Beacon ID is at radio packet byte 2 = frame offset 12** (`payload(2)` once stored in the payload register array below).

Relationship to the LH1-H Bring-Up Frame Detector

The LH1-H `frame_detector` locked on `0x7E`, counted to 43 bytes, and exposed Beacon ID and `new_frame` — but did not check the length byte, did not verify any CRC, and had no `REJECT` state. `pkt_parser` is the production replacement: it adds explicit length verification (this handout), CRC-8 and CRC-16 checks (LH2-E), a `REJECT` path with a pulse output, and deterministic re-synchronization after a bad frame.

4 Procedure

4.1 Step 1 — Remove the LH1-H `frame_detector`

1. In the Sources panel, right-click `frame_detector.vhd` → **Remove File from Project**. **Tick** "Also delete the file from disk." This file is never used again — `pkt_parser` is a strict superset.

2. `u_det` in the hierarchy will show a red "!" marker. You will replace it with a `u_parser` instance in Step 3.

4.2 Step 2 — Create `pkt_parser.vhd`

1. Sources panel: + → **Add or create design sources** → **Create File**. File type **VHDL**, name `pkt_parser`. Dismiss the Define Module dialog.

2. Open the new file, replace its contents with the listing below, and save.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity pkt_parser is
    port (
        clk           : in  std_logic;
        rst_n         : in  std_logic;
        -- byte_fifo read interface
        fifo_empty    : in  std_logic;
        fifo_data     : in  std_logic_vector(7 downto 0);
        fifo_rd_en    : out std_logic;
        -- Backpressure from feature extractor (LH2-F). Stub to '0' for
           now.
        feat_busy     : in  std_logic;
```

```
        -- Parsed outputs (stable on the cycle frame_valid is high)
        metadata      : out std_logic_vector(63 downto 0);   -- 8 bytes, byte 0 in LSB
        payload       : out std_logic_vector(255 downto 0);  -- 32 bytes, byte 0 in LSB
        rx_crc        : out std_logic_vector(7 downto 0);    -- captured CRC-8 (frame byte 42)
        frame_valid   : out std_logic;                        -- 1-cycle pulse, good frame
        frame_reject  : out std_logic                         -- 1-cycle pulse, bad frame
    );
end pkt_parser;

architecture rtl of pkt_parser is
    type state_t is (S_HUNT, S_REQ, S_WAIT, S_CAPTURE, S_VALID, S_REJECT);
    signal state    : state_t := S_HUNT;
    signal byte_idx : integer range 0 to 63 := 0;

    signal meta_r    : std_logic_vector(63 downto 0)  := (others => '0');
    signal payload_r : std_logic_vector(255 downto 0) := (others => '0');
    signal crc_r     : std_logic_vector(7 downto 0)   := (others => '0');
begin
    metadata <= meta_r;
    payload  <= payload_r;
    rx_crc   <= crc_r;

    process (clk) begin
        if rising_edge(clk) then
            frame_valid  <= '0';
            frame_reject <= '0';
            fifo_rd_en   <= '0';

            if rst_n = '0' then
                state    <= S_HUNT;
                byte_idx <= 0;
            else
                case state is
                    when S_HUNT =>
                        -- Wait for FIFO data and for feature extractor to be ready.
                        if fifo_empty = '0' and feat_busy = '0' then
                            fifo_rd_en <= '1';
                            state      <= S_WAIT;
                            byte_idx   <= 0;
                        end if;
                    when S_REQ =>
                        if fifo_empty = '0' then
                            fifo_rd_en <= '1';
                            state      <= S_WAIT;
                        end if;
                    when S_WAIT =>
                        state <= S_CAPTURE;             -- registered FIFO output appears
```

```
                    when S_CAPTURE =>
                        if byte_idx = 0 then
                            -- Looking for the start delimiter.
                            if fifo_data = x"7E" then
                                byte_idx <= 1;
                                state    <= S_REQ;
                            else
                                state <= S_HUNT;
                            end if;
                        elsif byte_idx = 1 then
                            -- Length byte must be 0x29.
                            if fifo_data = x"29" then
                                byte_idx <= 2;
                                state    <= S_REQ;
                            else
                                state <= S_REJECT;
                            end if;
                        elsif byte_idx >= 2 and byte_idx <= 9 then
                            -- Metadata bytes 0..7 (frame offsets 2..9).
                            meta_r(((byte_idx - 2) * 8 + 7) downto ((byte_idx - 2) * 8))
                                <= fifo_data;
                            byte_idx <= byte_idx + 1;
                            state    <= S_REQ;
                        elsif byte_idx >= 10 and byte_idx <= 41 then
                            -- Radio packet bytes 0..31 (frame offsets 10..41).
                            payload_r(((byte_idx - 10) * 8 + 7) downto ((byte_idx - 10) * 8))
                                <= fifo_data;
                            if byte_idx = 41 then
                                byte_idx <= 42;
                                state    <= S_REQ;
                            else
                                byte_idx <= byte_idx + 1;
                                state    <= S_REQ;
                            end if;
                        elsif byte_idx = 42 then
                            -- Trailing CRC-8 byte. CRC verification is added in LH2-E;
                            -- for now we simply capture and accept.
                            crc_r <= fifo_data;
                            state <= S_VALID;
                        end if;
                    when S_VALID =>
                        frame_valid <= '1';
                        state       <= S_HUNT;
                        byte_idx    <= 0;
                    when S_REJECT =>
                        frame_reject <= '1';
                        state        <= S_HUNT;
                        byte_idx     <= 0;
                end case;
            end if;
```

```
        end if;
    end process;
end rtl;
```

Listing 1: `pkt_parser.vhd` — 43-byte frame parser. Length-checked in this handout; CRC-checked in LH2-E. Exposes `metadata`, `payload`, and `rx_crc` stable on `frame_valid`.

4.3 Step 3 — Update `ingest_top.vhd` to use `pkt_parser`

The LH1-H `ingest_top` instantiated `u_det` (`frame_detector`). Replace it with `u_parser` (`pkt_parser`) and drive the `seg7_driver` from the parser's payload byte 2 (Beacon ID) and `frame_valid`. **Note**: the LH1-H `seg7_driver` entity itself is unchanged — only its source signals move from `frame_detector` to `pkt_parser`.

Open `ingest_top.vhd`, replace its contents with the listing below, and save.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ingest_top is
    port (
        CLK100MHZ    : in  std_logic;
        CPU_RESETN   : in  std_logic;
        uart_rx_pin  : in  std_logic;                      -- PMOD JA pin 1 (C17)
        UART_RXD_OUT : out std_logic;                      -- onboard USB-UART (LH2-A)
        CA, CB, CC, CD, CE, CF, CG : out std_logic;
        DP           : out std_logic;
        AN           : out std_logic_vector(7 downto 0);
        LED          : out std_logic_vector(15 downto 0)
    );
end ingest_top;

architecture rtl of ingest_top is
    signal rst_n      : std_logic;

    signal rx_data    : std_logic_vector(7 downto 0);
    signal rx_strobe  : std_logic;

    signal fifo_data  : std_logic_vector(7 downto 0);
    signal fifo_rd_en : std_logic;
    signal fifo_empty : std_logic;
    signal fifo_full  : std_logic;

    signal metadata     : std_logic_vector(63 downto 0);
    signal payload      : std_logic_vector(255 downto 0);
    signal rx_crc       : std_logic_vector(7 downto 0);
    signal frame_valid  : std_logic;
    signal frame_reject : std_logic;

    -- Alias kept so the seg7_driver port map still reads "new_frame => new_frame"
    -- (the LH1-H block-diagram name); the parser's frame_valid is its source.
```

```
    signal new_frame  : std_logic;
    signal beacon_id  : std_logic_vector(7 downto 0);
    signal seg_bus    : std_logic_vector(6 downto 0);
begin
    rst_n <= CPU_RESETN;

    u_uart : entity work.uart_rx
        port map (clk => CLK100MHZ, rst_n => rst_n,
                  rx => uart_rx_pin,
                  rx_data => rx_data, rx_strobe => rx_strobe);

    u_fifo : entity work.byte_fifo
        port map (clk => CLK100MHZ, rst_n => rst_n,
                  wr_en => rx_strobe, wr_data => rx_data,
                  rd_en => fifo_rd_en, rd_data => fifo_data,
                  empty => fifo_empty, full => fifo_full);

    u_parser : entity work.pkt_parser
        port map (clk => CLK100MHZ, rst_n => rst_n,
                  fifo_empty => fifo_empty, fifo_data => fifo_data,
                  fifo_rd_en => fifo_rd_en,
                  feat_busy => '0',                          -- LH2-F replaces with real signal
                  metadata => metadata, payload => payload,
                  rx_crc => rx_crc,
                  frame_valid => frame_valid, frame_reject => frame_reject);

    -- Beacon ID = radio packet byte 2 = payload bytes (15 downto 8) in the
    -- packed payload vector (byte 0 in LSB, byte 2 spans bits 23..16).
    beacon_id <= payload(23 downto 16);
    new_frame <= frame_valid;

    u_seg : entity work.seg7_driver
        port map (clk => CLK100MHZ, rst_n => rst_n,
                  beacon_id => beacon_id, new_frame => new_frame,
                  seg => seg_bus, dp => DP, an => AN);

    CA <= seg_bus(0);
    CB <= seg_bus(1);
    CC <= seg_bus(2);
    CD <= seg_bus(3);
    CE <= seg_bus(4);
    CF <= seg_bus(5);
    CG <= seg_bus(6);

    LED(0)            <= rx_strobe;
    LED(1)            <= frame_valid;
    LED(2)            <= fifo_empty;
    LED(3)            <= fifo_full;
    LED(4)            <= frame_reject;
    LED(15 downto 5)  <= (others => '0');

```

```
    -- Passthrough: forward the raw PMOD JA1 stream out the onboard USB-UART
    -- (added in LH2-A so the host can capture .bin files via the same USB
    -- cable that programs the board). Both ends are 115200 8N1.
    UART_RXD_OUT <= uart_rx_pin;
end rtl;
```

Listing 2: `ingest_top.vhd` (Module 2 LH2-D revision) — `frame_detector` replaced by `pkt_parser`; `seg7_driver` now displays Beacon ID from `payload(15 downto 8)` (which is radio packet byte 2 = frame offset 12).

4.4 Step 4 — Re-synthesize and confirm hardware still runs

1. **Confirm `ingest_top` is set as the Design Sources top.** In the Sources panel's *Design Sources* tab, `ingest_top` must appear in **bold**. After Module 2 simulation runs, the testbench you set as top in Simulation Sources is unrelated, but Vivado occasionally also reassigns the design top during a hierarchy shuffle (such as the `frame_detector` removal in Step 1). If anything other than `ingest_top` is bold under Design Sources, right-click `ingest_top` → **Set as Top** before continuing.

2. Run synthesis. Expect zero errors. The `u_parser` red "!" marker should clear.

3. Re-program the board. The Beacon ID display should still update; `LED[1]` (`frame_valid`) should pulse at the per-frame cadence; `LED[4]` (`frame_reject`) should stay dark when the Threat is emitting clean frames.

[Figure placeholder]

`LH2D_parser_fsm.png`

Packet parser Finite State Machine (FSM) with the full transition set. Each labeled arc shows the byte or condition that fires the transition.

Figure 1: Packet parser FSM with the full transition set. Each labeled arc shows the byte or condition that fires the transition.

4.5 Step 5 — Parser testbench

Add a simulation source `tb_pkt_parser`. The testbench drives a small in-memory First In, First Out (FIFO) stub (rather than the real `byte_fifo`) so the test runs in microseconds. It pre-loads three frames: valid, rejected (wrong length byte), valid.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tb_pkt_parser is
end tb_pkt_parser;

architecture sim of tb_pkt_parser is
    constant CLK_PERIOD : time := 10 ns;

    signal clk          : std_logic := '0';
    signal rst_n        : std_logic := '0';

    -- Behavioural byte source acts as a 1-deep FIFO.
    signal src_data     : std_logic_vector(7 downto 0) := (others => '0');
    signal src_empty    : std_logic := '1';
    signal rd_en        : std_logic;
    signal pending_byte : std_logic_vector(7 downto 0) := (others => '0');
    signal has_pending  : std_logic := '0';

    signal metadata     : std_logic_vector(63 downto 0);
    signal payload      : std_logic_vector(255 downto 0);
    signal rx_crc       : std_logic_vector(7 downto 0);
    signal frame_valid  : std_logic;
    signal frame_reject : std_logic;

    signal valid_count  : integer := 0;
    signal reject_count : integer := 0;
    signal last_id      : std_logic_vector(7 downto 0) := (others => '0');

    -- Build a 43-byte frame with a given Beacon ID, length byte, and a
    -- placeholder CRC (parser doesn't check CRC until LH2-E).
    type byte_array_t is array (natural range <>) of std_logic_vector(7 downto 0);

    function make_frame (beacon : std_logic_vector(7 downto 0);
                         len    : std_logic_vector(7 downto 0)) return byte_array_t is
        variable f : byte_array_t(0 to 42) := (others => (others => '0'));
    begin
        f(0)  := x"7E";
        f(1)  := len;
        -- bytes 2..9 metadata, leave zero
        -- byte 10 = radio packet byte 0, byte 12 = radio packet byte 2 = Beacon ID
        f(12) := beacon;
        -- bytes 13..41 payload, leave zero
        f(42) := x"AA";                                  -- placeholder CRC
        return f;
    end function;

    procedure push_frame (signal pend : out std_logic_vector(7 downto 0);
```

```
                            signal hp    : out std_logic;
                            signal r     : in  std_logic;
                            frame        : byte_array_t) is
    begin
        for i in frame'range loop
            pend <= frame(i);
            hp   <= '1';
            -- Wait until parser consumes this byte.
            wait until rising_edge(clk) and r = '1';
            hp <= '0';
            wait until rising_edge(clk);
        end loop;
    end procedure;
begin
    clk <= not clk after CLK_PERIOD / 2;

    -- Behavioural byte source: when has_pending is high, src_empty is low and
    -- src_data is the pending byte. The DUT pulses rd_en; the stimulus
    -- process clears has_pending in response.
    src_empty <= not has_pending;
    src_data  <= pending_byte;

    dut : entity work.pkt_parser
        port map (clk => clk, rst_n => rst_n,
                  fifo_empty => src_empty, fifo_data => src_data,
                  fifo_rd_en => rd_en,
                  feat_busy => '0',
                  metadata => metadata, payload => payload, rx_crc => rx_crc,
                  frame_valid => frame_valid, frame_reject => frame_reject);

    process (clk) begin
        if rising_edge(clk) then
            if frame_valid = '1' then
                valid_count <= valid_count + 1;
                last_id     <= payload(23 downto 16);    -- radio byte 2 = Beacon ID
            end if;
            if frame_reject = '1' then
                reject_count <= reject_count + 1;
            end if;
        end if;
    end process;

    process
    begin
        wait for 100 ns;
        rst_n <= '1';
        wait for 200 ns;

        -- Frame 1: valid, Beacon ID = 0x01.
        push_frame(pending_byte, has_pending, rd_en, make_frame(x"01", x"
```

```
            29"));
        wait for 500 ns;
        assert valid_count = 1
            report "frame 1 should have produced frame_valid" severity failure;
        assert last_id = x"01" report "frame 1 Beacon ID mismatch" severity failure;

        -- Frame 2: bad length byte.
        push_frame(pending_byte, has_pending, rd_en, make_frame(x"02", x"30"));
        wait for 500 ns;
        assert reject_count = 1
            report "frame 2 should have produced frame_reject" severity failure;

        -- Frame 3: valid, Beacon ID = 0x02.
        push_frame(pending_byte, has_pending, rd_en, make_frame(x"02", x"29"));
        wait for 500 ns;
        assert valid_count = 2
            report "frame 3 should have produced second frame_valid" severity failure;
        assert last_id = x"02" report "frame 3 Beacon ID mismatch" severity failure;

        report "tb_pkt_parser PASS" severity note;
        std.env.finish;
    end process;
end sim;
```

Listing 3: `tb_pkt_parser.vhd` — three-frame stimulus drives a behavioral byte source. Asserts two `frame_valid` pulses and one `frame_reject` pulse, and verifies the captured Beacon ID on each valid frame.

Set `tb_pkt_parser` as the simulation top (right-click in Sources → Simulation Sources → **Set as Top**), launch behavioral simulation, then in the Tcl console type `run 20 us` (once) and press Enter. Three frames at ∼1.3 μs each plus inter-frame waits keeps the test under 10 μs of simulated time; 20 μs gives margin. Confirm `tb_pkt_parser PASS` prints in the Tcl console.

5 Verification Checklist

- ☐ LH1-H `frame_detector.vhd` deleted from project and disk.
- ☐ `pkt_parser.vhd` added; `ingest_top.vhd` updated to instantiate `u_parser` and drive `seg7_driver` from `payload(23 downto 16)`.
- ☐ Synthesis produces zero errors and the seven-segment display still shows live Beacon IDs on hardware.
- ☐ `tb_pkt_parser` prints `PASS`: two `frame_valid` pulses, one `frame_reject` pulse, Beacon IDs match the injected values.

[Figure placeholder]

`LH2D_tb_waveform.png`

Testbench waveform showing three frames: valid, rejected, valid. The `frame_valid` and `frame_reject` pulses occur on the correct cycles.

Figure 2: Testbench waveform showing three frames: valid, rejected, valid. The `frame_valid` and `frame_reject` pulses occur on the correct cycles.

- ☐ `LED[4]` (`frame_reject`) stays dark on a clean live capture; `LED[1]` (`frame_valid`) pulses at the ping-pong cadence.

Common Pitfalls

- **Beacon ID slice:** the radio packet's byte 2 is at frame offset 12. In the packed `payload` vector with byte 0 in the LSB, byte 2 occupies bits `23 downto 16`. Driving `seg7_driver.beacon_id` from `payload(7 downto 0)` would display the radio packet's byte 0 (a header field), not the Beacon ID.
- **Sampling `rd_data` on the same cycle as `rd_en`:** `byte_fifo` has a one-cycle registered read, so the `S_REQ` → `S_WAIT` → `S_CAPTURE` sequence is required. Removing `S_WAIT` captures the *previous* byte every time.
- **`feat_busy` grounded forever:** fine for this handout, but LH2-F replaces `'0'` with the real signal from the feature extractor. If you skip that step, the parser will overrun the extractor.

Known limitation: `feat_busy` is only checked at frame boundaries

The `S_HUNT` state gates entry to a new frame on `feat_busy = '0'`, but `S_REQ`/`S_WAIT`/`S_CAPTURE` do *not* re-check `feat_busy` mid-frame. Once a frame starts parsing, the parser commits to draining all 43 bytes from the FIFO regardless of downstream readiness. This is benign at current latencies: the LH2-F feature extractor's read–modify–write completes in 3 `sysclk` cycles (∼30 ns), while a single Universal Asynchronous Receiver/Transmitter (UART) byte takes ∼86.8 μs — so the extractor is always idle long before the parser reaches the next frame boundary. If a future Module (e.g. a learned scoring kernel that takes longer than one byte-time) ever extends the extractor's busy window across frame boundaries, this gating must move into `S_REQ` as well. Do not rely on mid-frame backpressure.

Lab Handout 2-D Deliverable

1. `pkt_parser.vhd` and updated `ingest_top.vhd`.

2. `tb_pkt_parser.vhd`.
3. Tcl console transcript showing `tb_pkt_parser PASS`.
4. Waveform screenshot annotating the `VALID/REJECT/VALID` sequence and the Beacon ID on each `frame_valid` cycle.

Glossary

CRC	Cyclic Redundancy Check	1, 2, 5
FIFO	First In, First Out	7, 11
FPGA	Field-Programmable Gate Array	1
FSM	Finite State Machine	7
RSSI	Received Signal Strength Indicator	1
RTL	Register-Transfer Level	1
SNR	Signal-to-Noise Ratio	1
UART	Universal Asynchronous Receiver/Transmitter	11